Network Management — Academic Year 2025/2026

ClickHouse Ingestion and Behavioural Analysis (CHIBA)

Author: Giuseppe Nozza

Course Instructor: Prof. Luca Deri

November 5, 2026

Contents

1 Project Description **1**

1.1 Objectives . . . 1

1.2 From Probe Capture to Binary Export . . . 1

1.2.1 CLI Parsing and Configuration Management . . . 1

1.2.2 Managing `softflowd` . . . 2

1.2.3 Parsing NetFlow v9 Records . . . 3

1.2.4 Producer-Consumer Pattern and Double Buffering . . . 4

1.2.5 HTTP Streaming . . . 5

1.2.6 Memory Cleanup . . . 6

1.3 ClickHouse Schema and Data Pipeline . . . 8

1.3.1 Base Table Schema . . . 8

1.3.2 Data Aggregation and Materialized Views . . . 9

1.3.3 Logical Views Layer . . . 9

1.4 Monitoring and Alerting . . . 9

1.4.1 Traffic and Telemetry Panels . . . 10

1.4.2 Detection Rules, Thresholds, and Severity . . . 16

2 Prerequisites and Execution Instructions **17**

2.1 System Requirements and Dependency Installation . . . 17

2.1.1 Development Toolchain, Libraries, and Capture Probe . . . 17

2.1.2 ClickHouse Installation (Server and Client) . . . 18

2.1.3 Grafana and ClickHouse Plugin Installation . . . 18

2.1.4 Test Traffic Generation Tools . . . 18

2.2 Compiling the CHIBA Daemon . . . 18

2.3 Database Initialization and Management . . . 19

2.4 Daemon Configuration and Execution . . . 19

2.5 Grafana Dashboard Configuration . . . 19

3 Testing **20**

3.1 Fast Port Scan . . . 20

3.2 Network Sweep (Horizontal Scan) . . . 20

3.3 Distributed Flood / DDoS . . . 21

3.4 Anomalous Egress Volume (Data Exfiltration) . . . 22

3.5 Dual-Stack IPv6 Support from PCAP Trace . . . 23

3.6 Slow Port Scan (24-Hour Window) 24

4 Conclusions and Future Work **24**

5 References and Bibliography **25**

1 Project Description

1.1 Objectives

The purpose of this telemetry tool is to analyze the behavior of network flows in real time and from PCAP files in order to detect Distributed Denial-of-Service (DDoS) attacks, data exfiltration, horizontal network scans, and both fast and slow vertical port scans.

1.2 From Probe Capture to Binary Export

Figure 1 illustrates the internal process and thread architecture of the CHIBA daemon, from initial bootstrap and packet capture to bulk binary ingestion into ClickHouse.

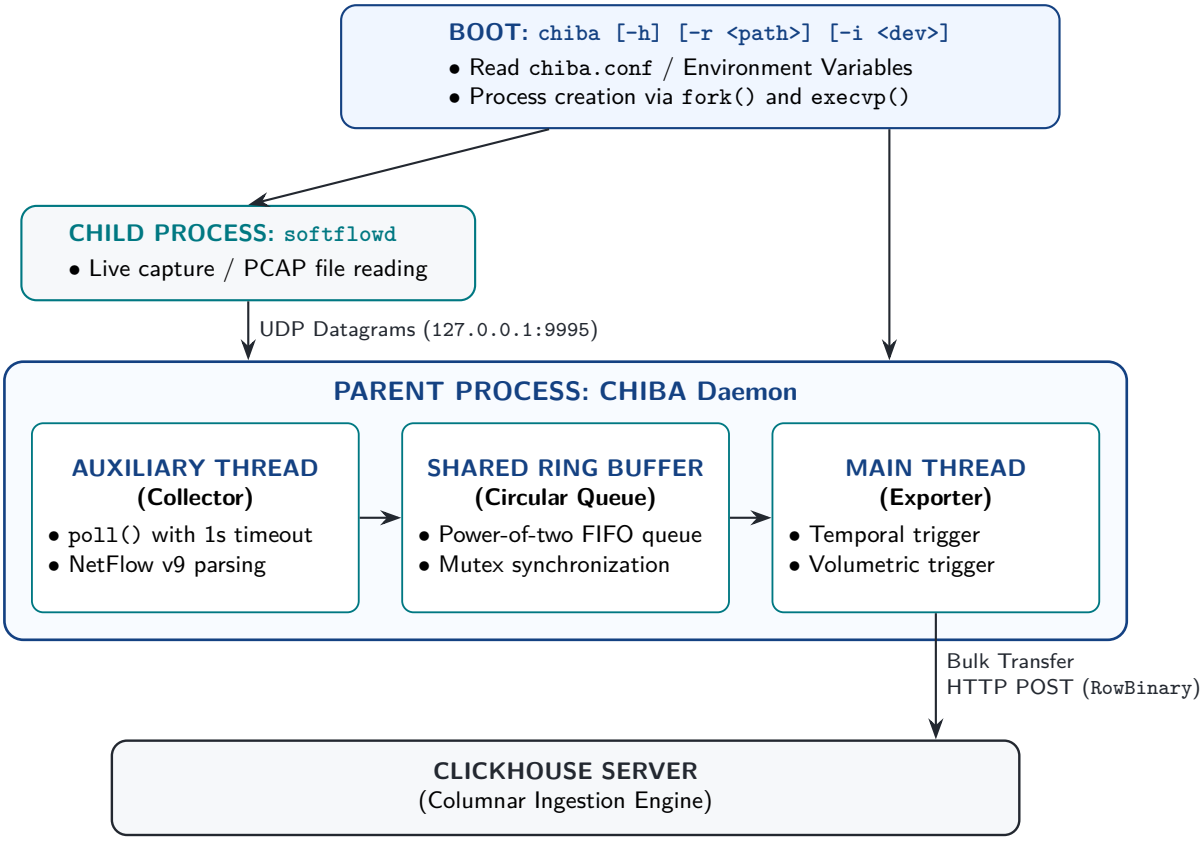


Figure 1: End-to-end architecture of the CHIBA daemon and its interaction with `softflowd` and ClickHouse.

1.2.1 CLI Parsing and Configuration Management

Command-line interface (CLI) parsing is implemented using the POSIX `getopt()` function. The network traffic source is defined during this stage; live capture mode on a network interface and offline capture mode from a PCAP file are mutually exclusive.

The parameters required to build the URL for ClickHouse's HTTP ingestion interface are extracted from the `chiba.conf` configuration file via the `load_config()` routine. These parameters can be overridden at runtime through dedicated environment variables. Below is the output

produced when running `chiba` with the help flag (`-h`). Details regarding the execution path are covered in Section 2.

CLI Help Output

```
./bin/chiba -h
[INFO] chiba.conf read correctly.
Usage: chiba [-h] [-r <path>] [-i <device>]

Options:
  -h               Print help
  -r <path>        Static PCAP file path
  -i <device>      Live network interface name

Environment Variables for ClickHouse HTTP interface (override chiba.conf):
  CH_HOST          Host (default: 127.0.0.1)
  CH_DB            Database (default: chiba)
  CH_TABLE         Destination table (default: ingest_flows)
```

1.2.2 Managing `softflowd`

The `softflowd` probe is instantiated by `chiba` via `fork()` and `execvp()`, running in the background as a concurrent child process while the parent process proceeds with daemon initialization.

The child process's `stdout` and `stderr` descriptors are redirected to `/dev/null` to prevent interference with CHIBA's logs.

NetFlow v9 records are exported to the collector thread via UDP datagrams (on `127.0.0.1:9995`). UDP is chosen for clear performance reasons, eliminating the connection, handshake, and acknowledgment overhead typical of TCP in favor of maximum throughput.

The argument array passed to `execvp()` differentiates capture parameters and flow export policies according to the operating mode:

- **Live Mode:** The probe captures traffic in real time from the specified network interface. Flow timeouts are balanced for continuous monitoring: each flow has a maximum lifetime of 60 seconds before export; UDP inactivity timeout is set to a maximum of 30 seconds, while inactive TCP flows are exported within 5 minutes. This trade-off prevents saturation of `softflowd`'s active flow table and avoids excessive fragmentation of long-lived connections, ensuring up-to-date data reaches the collector without generating unnecessary network overhead.
- **Offline Mode:** The probe sequentially processes a static PCAP trace. Time constraints (maximum lifetime and emission intervals) are reduced to 1 second, and the flow table is constrained to its minimum size. This forces immediate eviction of every flow record toward the UDP socket, enabling rapid scanning of the packet dump without the pauses and idle intervals of the original traffic.

Immediately after `fork()`, the parent process performs a non-blocking check on the probe's status via:

Non-Blocking Child Process Check (C)

```
if (waitpid(pid, &status, WNOHANG) > 0) { ... }
```

Using the `WNOHANG` flag avoids blocking and allows immediate detection of any startup failure in `softflowd` (for instance, due to a non-existent interface, insufficient privileges, or an invalid PCAP file path).

Finally, safe termination of the child process is handled by the daemon upon shutdown (or upon completion of PCAP file processing): CHIBA sends a `SIGTERM` signal to the recorded PID and invokes `waitpid()` in blocking mode to reap the process exit status, preventing zombie processes from lingering in the system.

1.2.3 Parsing NetFlow v9 Records

The responsibility of the collector thread is to receive the UDP packets sent by the probe and transform them into structured records ready for the database. Adopting NetFlow v9 entails additional complexity compared to rigid formats such as NetFlow v5: packets do not have a fixed structure, but instead decouple schema definitions from actual flow data.

At startup and at regular intervals, the probe transmits schema packets (**Template FlowSets**) that declare the composition of a flow: which fields are present, their sequence order, and their size in bytes. The parser stores these definitions in a static internal table for the entire duration of execution, without resorting to dynamic heap memory allocations.

When packets containing actual traffic (**Data FlowSets**) arrive, the parser consults the cached schema and extracts field values directly from the binary payload. Three primary operations are performed during this stage:

- **Byte Decoding and Endianness Conversion:** Traffic counters and network ports are extracted according to their actual byte length (ranging from 1 to 8 bytes) and converted from network byte order (big-endian) to the host architecture's native byte order.
- **Address Space Unification:** To handle IPv4 and IPv6 simultaneously with a single unified logic, IPv4 addresses are mapped into the standard 128-bit `IPv4-mapped` IPv6 format (`::ffff:a.b.c.d`). This guarantees that every flow has an identical Layer-3 representation regardless of the underlying IP protocol version.
- **Normalization into Internal Format:** Extracted values are written into a compact structure derived from standard Cisco telemetry fields:

Internal Flow Record Structure (`include/flow.h`)

```
typedef struct flow_data {
  u_int32_t start_time;
  u_int32_t end_time;

  u_int8_t  srcaddr[16];
  u_int8_t  dstaddr[16];

  u_int32_t dPkts;
  u_int32_t dOctets;
  u_int16_t srcport;
  u_int16_t dstport;
  u_int8_t  prot;
} __attribute__((packed)) flow_data;
```

Using the `__attribute__((packed))` compiler attribute prevents the compiler from inserting padding bytes for structure alignment. Consequently, the structure has a net byte size that matches the exact binary sequence expected by ClickHouse for direct HTTP insertion, enabling

bulk transfer of the entire memory block in `RowBinary` format without intermediate serialization steps.

NetFlow v9 Parsing Fixes and PCAP Trace Requirements

- **Complete FlowSet Scanning:** In the NetFlow v9 specification (RFC 3954), `header->count` indicates the total number of records (both template and data records) rather than the number of FlowSets. To prevent premature loop termination that previously caused templates and data records to be dropped in multi-FlowSet datagrams (such as in `DHCPv6.pcap`), packet scanning is driven by the total buffer length (`while (offset + sizeof(...) <= length)`).
- **Encapsulation Requirement for `softflowd` (`DLT_NULL` Endianness):** On `x86_64` architectures, `softflowd` requires packets to use standard Ethernet encapsulation (`DLT_EN10MB`). In packet dumps captured on BSD loopback interfaces in big-endian order (`DLT_NULL`, e.g., `snmp_usm.pcap`), the 4-byte Address Family header is not byte-swapped by `libpcap`, causing `softflowd` to discard all traffic as `non-IP`. To allow `softflowd` to process such captures, they must be normalized to Ethernet using `tcprewrite --dlt=enet`. Because the original capture lacks Ethernet frames, dummy source and destination MAC addresses must be supplied explicitly:

  ```
  tcprewrite --dlt=enet \
    --enet-dmac=00:11:22:33:44:55 \
    --enet-smac=aa:bb:cc:dd:ee:ff \
    -i ./pcap/snmp_usm.pcap \
    -o ./pcap/snmp_usm_enet.pcap
  ```

- **Temporal Alignment to Present Time:** In PCAP mode, CHIBA assigns the current system timestamp (`time(NULL)`) to ingested flows. This prevents the creation of historical partitions in ClickHouse and allows immediate visualization of plots within Grafana's default "Today" dashboards.

1.2.4 Producer-Consumer Pattern and Double Buffering

Decoupling high-frequency UDP packet reception from database persistence relies on the producer-consumer pattern, implemented through a shared circular queue (`ring_buffer`). The auxiliary collector thread acts as the producer by inserting decoded flow records, while the main thread acts as the consumer by extracting accumulated records to send to ClickHouse.

Access to the shared resource is governed by a single `mutex`, deliberately omitting condition variables. Condition variables are unnecessary because the consumer does not need to wake up on every single insertion, nor does the producer wait for the queue to drain: the main thread evaluates extraction based on asymmetric triggers (either exceeding a volumetric threshold of accumulated records or reaching a maximum elapsed time interval). Eliminating calls to `pthread_cond_signal` and `pthread_cond_wait` removes context-switch overhead and minimizes lock holding time.

During bulk transfer invocations, memory management leverages a *double buffering* strategy: while a batch of flows is isolated and transmitted over HTTP, the producer has the remaining free half of the ring buffer available to continue recording incoming flows, preventing packet drops or overwrites of unconsumed data in the queue.

Ring Buffer Implementation Notes The circular queue does not implement a traditional single-element extraction routine (`pop` element by element). Extraction occurs exclusively in contiguous blocks during bulk transfer, eliminating per-record synchronization overhead and minimizing contention on the shared mutex.

Advancing the circular indices leverages a buffer size strictly constrained to a power of two ($2^{15} = 32768$). This algebraic property makes it possible to compute wrapped indices without the modulo operator (`%`), applying a fast bitwise AND mask instead:

Bitwise Ring Buffer Index Advancement (C)

```
/* Single-record enqueue (Producer) */
rb->tail = (rb->tail + 1) & (rb->size - 1);

/* Batch head advancement by n records (Consumer) */
rb->head = (rb->head + n) & (rb->size - 1);
```

1.2.5 HTTP Streaming

The main thread manages flow transmission to ClickHouse by extracting accumulated records whenever the volumetric batch threshold (`EXPORT_BUFFER_SIZE = 16384`) is reached or the configured time interval (`EXPORT_MIN_TIME_MS = 5000` ms) expires.

Data extracted from the circular queue first passes through a linear export buffer (`export_buffer`). This intermediate staging buffer is necessary for two reasons: first, it linearizes memory whenever a batch of flows wraps around the physical boundary between the end and the beginning of the ring buffer; second, it allows the shared mutex to be released immediately after the memory copy operation, avoiding holding the lock during network I/O.

Flow Ingestion via `libcurl-easy` Adopting `libcurl`'s synchronous `easy` interface allows the exporter thread to manage the HTTP channel linearly across four sequential steps:

- **Endpoint Configuration:** The request targets the local ClickHouse instance by passing the SQL `INSERT` statement directly as a URL query parameter specifying the `RowBinary` format. This keeps the SQL command definition separate from the `POST` request body, which is reserved entirely for the raw binary data stream.
- **Payload Binding:** The linear buffer is attached directly to the `POST` request body along with the total byte count to transmit. Because the `RowBinary` format contains no headers, metadata, or delimiter characters, ClickHouse interprets the stream by reading memory sequentially and mapping it positionally (1:1) onto the columns of the destination table. This design eliminates all computational overhead associated with text formatting or serialization.
- **Transfer Execution:** Transmission occurs synchronously via `curl_easy_perform()`, blocking the exporter thread until the batch upload completes. Furthermore, `libcurl`'s connection pool automatically reuses the underlying TCP connection across consecutive batches, reducing overall latency.
- **Response Validation:** Upon completion of the transfer, the HTTP response returned by the DBMS is inspected to verify that the current batch was persisted successfully before resetting buffer pointers and proceeding with the next accumulation cycle.

1.2.6 Memory Cleanup

All dynamic heap memory allocations and system resource acquisitions are strictly confined to the initial configuration and runtime setup phase. During this startup phase, the parent process validates CLI arguments, opens the UDP socket descriptor, spawns the child probe process, drops user privileges to `nobody`, allocates the ring and export buffers, initializes the `libcurl` context, and launches the auxiliary collector thread.

During the steady-state monitoring phase—after the auxiliary thread has been launched—no calls to `malloc()` or `free()` are ever executed.

Resource deallocation mirrors the acquisition sequence through a strict Last-In, First-Out (LIFO) cascade. When the daemon stops—triggered either by `SIGINT`/`SIGTERM` signal handlers or by child process termination at the end of a PCAP trace—the program exits its main loop and deallocates components in reverse order of allocation. Below is the cleanup sequence directly from `main.c`:

Graceful LIFO Teardown (`src/main.c`)

```
/* LIFO cleanup */
curl_easy_cleanup(curl);
curl_global_cleanup();
export_buffer_destroy(eb);
ring_buffer_destroy(rb);

kill(pid, SIGTERM);
waitpid(pid, NULL, 0);
close(sockfd);

puts("[INFO] Shutting down");
return (EXIT_SUCCESS);
```

If an initialization or allocation error occurs in any component during startup, resources acquired up to that point are unwound via an ordered chain of `goto` labels:

Cascading Error Unwinding via `goto` (`src/main.c`)

```
/* LIFO cascade cleanup */
err_curl_easy:
    curl_easy_cleanup(curl);
err_curl_global:
    curl_global_cleanup();
err_eb:
    export_buffer_destroy(eb);
err_rb:
    ring_buffer_destroy(rb);
err_child:
    kill(pid, SIGTERM);
    waitpid(pid, NULL, 0);
err_socket:
    close(sockfd);

fprintf(stderr, "[ERROR] Shutting down: see error above\n");
return (EXIT_FAILURE);
```

Valgrind Verification Memory management correctness and complete resource deallocation were verified using Valgrind by subjecting the daemon to two distinct operational scenarios with

explicit tracking of heap blocks and file descriptors.

The first test verifies the full program lifecycle on a static PCAP trace containing IPv6 traffic:

Valgrind Verification — Offline PCAP Mode

```
valgrind --leak-check=full --show-leak-kinds=all --track-fds=yes \
  ./bin/chiba -r ./pcap/DHCPv6.pcap

==16921== FILE DESCRIPTORS: 4 open (3 std) at exit.
==16921== Open AF_UNIX socket 12: <unknown>
==16921==    <inherited from parent>
==16921==
==16921== HEAP SUMMARY:
==16921==     in use at exit: 0 bytes in 0 blocks
==16921==   total heap usage: 4,293 allocs, 4,293 frees, 2,842,787 bytes allocated
==16921==
==16921== All heap blocks were freed -- no leaks are possible
==16921==
==16921== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

The report confirms a 1:1 match between allocations and frees. File descriptor analysis confirms proper closure of the receiving UDP socket and redirected pipes; the single non-standard open descriptor is inherited from the parent shell process, ruling out any resource leak inside the daemon.

The second test verifies that the program also passes all memory and descriptor checks when interrupted asynchronously via a signal during live capture:

Valgrind Verification — Live Capture Mode with Asynchronous Interrupt

```
valgrind --leak-check=full --show-leak-kinds=all --track-fds=yes ./bin/chiba -i lo

==17519==
==17519== FILE DESCRIPTORS: 3 open (3 std) at exit.
==17519==
==17519== HEAP SUMMARY:
==17519==     in use at exit: 0 bytes in 0 blocks
==17519==   total heap usage: 4,315 allocs, 4,315 frees, 2,858,557 bytes allocated
==17519==
==17519== All heap blocks were freed -- no leaks are possible
==17519==
==17519== For lists of detected and suppressed errors, rerun with: -s
==17519== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

1.3 ClickHouse Schema and Data Pipeline

Figure 2 illustrates the multi-stage ClickHouse storage and analytical pipeline, from raw binary flow ingestion to pre-aggregated state tables, logical merge views, and Grafana visualization.

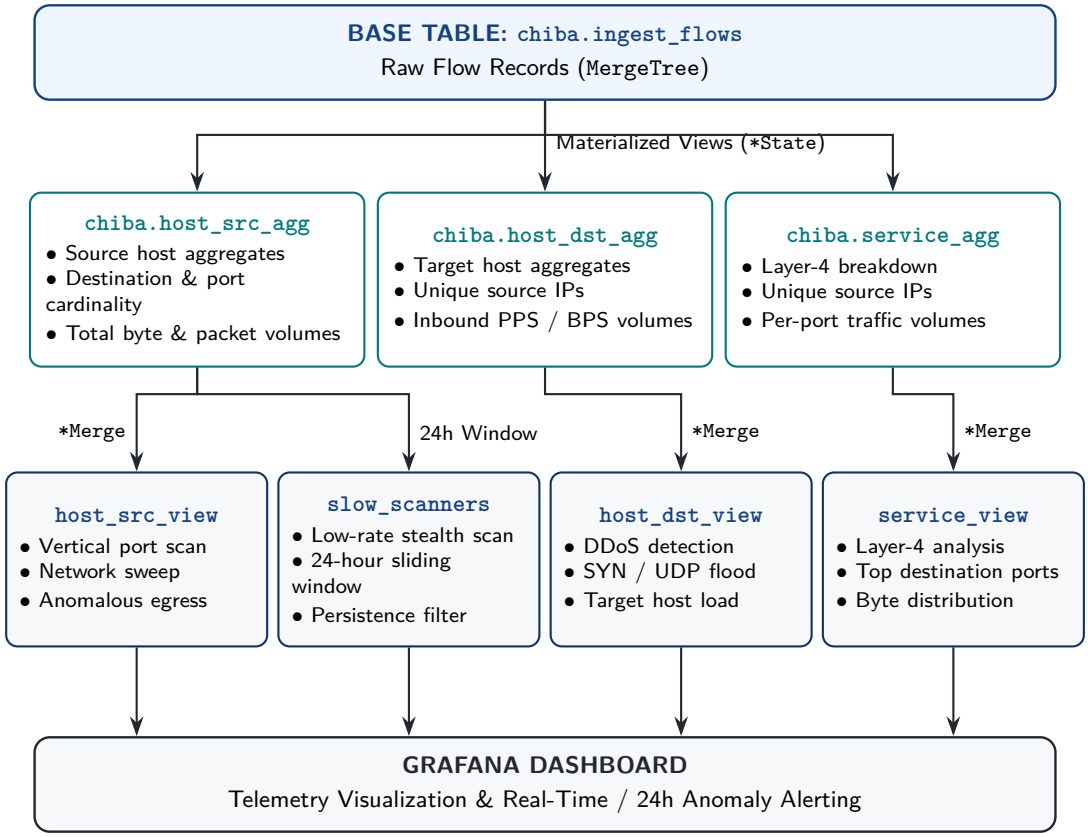


Figure 2: ClickHouse storage hierarchy, continuous Materialized View pre-aggregation, and logical presentation views.

1.3.1 Base Table Schema

The `chiba.ingest_flows` table stores raw flow records and serves as the data source for all materialized views in the analytical pipeline. Its architectural design choices are as follows:

- Column data types have a strict one-to-one correspondence with the fields of the C `flow_data` structure described in Section 1.2.3. As discussed in Section 1.2.5.1, the `RowBinary` ingestion format requires no decoding or type conversion.
- `ENGINE = MergeTree()`: Using ClickHouse's standard columnar engine organizes records on disk into parts that are compacted asynchronously in the background.
- `PARTITION BY toYYYYMMDD(toDateTime(START_TIME))`: Physically segments data on disk into daily partitions.
- `ORDER BY (START_TIME, SRCADDR, DSTADDR)`: Giving precedence to `START_TIME` optimizes range scans over time intervals, while including `SRCADDR` and `DSTADDR` in the primary sorting key accelerates filtering and grouping operations in ad-hoc inspection queries over raw flow data.

1.3.2 Data Aggregation and Materialized Views

To prevent telemetry and anomaly detection queries from scanning millions of raw flow records on every dashboard refresh, the pipeline interposes a continuous pre-aggregation layer built on three `AggregatingMergeTree` tables (`host_src_agg`, `host_dst_agg`, and `service_agg`) populated by three corresponding Materialized Views.

In ClickHouse, Materialized Views operate as write-time triggers: whenever a batch of rows is inserted into `ingest_flows`, the newly arrived block is intercepted directly in memory and processed according to a dual logic:

- **Temporal Discretization:** The expression `toStartOfMinute(toDateTime(START_TIME))` maps individual flow timestamps into fixed 1-minute time windows (`TIME_BUCKET`). This interval forms the base granularity upon which all downstream metrics are computed.
- **Intermediate State Persistence:** Rather than storing a final scalar number, the materialized views use ClickHouse's state combinator aggregate functions (`*State`). The resulting values are binary intermediate states that preserve the internal data structure (for example, the probabilistic HyperLogLog register for `uniqState` or the temporal bounds for `minState`/`maxState`) so they can be incrementally merged across arbitrary time ranges at query time.

The three materialized views partition network traffic across three independent analytical dimensions:

1. `host_src_mv`: Groups by (`TIME_BUCKET`, `SRCADDR`, `PROT`), tracking the dispersion of outbound connections (unique destination IP addresses and destination ports) to detect scans and anomalous outbound traffic.
2. `host_dst_mv`: Groups by (`TIME_BUCKET`, `DSTADDR`), quantifying converging flows and unique source IP addresses per target host to detect flood and DDoS attacks.
3. `service_mv`: Groups by (`TIME_BUCKET`, `DSTPORT`, `PROT`), aggregating traffic volume concentration across individual network services and target ports.

1.3.3 Logical Views Layer

Logical Views form the database presentation layer, exposing ready-to-query interfaces to Grafana panels. Within these views, ClickHouse's merge combinators (`*Merge`) are applied to finalize the binary aggregate states into scalar metrics. This shields dashboard queries from having to know the internal data structures of the aggregation engine.

Beyond 1:1 resolution of 1-minute buckets for standard telemetry, this layer enables dedicated analytical views such as `slow_scanners_view`, which re-aggregates 1-minute buckets across an extended 24-hour horizon to compute derived rate metrics and filter stealth anomalies directly within the DBMS.

1.4 Monitoring and Alerting

The visualization and alerting infrastructure is built on Grafana, connected to ClickHouse via the official ClickHouse datasource plugin. The `chiba` dashboard directs all queries to the logical views described above. Queries leverage Grafana's native time macros (specifically

`$__timeFilter(TIME_BUCKET)`), delegating all time-range filtering and columnar aggregation to ClickHouse.

1.4.1 Traffic and Telemetry Panels

Privacy Disclaimer

The screenshots and telemetry data shown in this report originate from a controlled laboratory test environment. No private or sensitive IP addresses attributable to real production infrastructures are displayed; visible hosts belong either to laboratory subnets (RFC 1918 / loopback) or to well-known public services and CDNs.

The Grafana dashboard is organized into four thematic rows:

1. Inbound Traffic Queries `chiba.host_dst_view` to monitor incoming load toward internal targets. It includes a table of the top ten destination hosts by volume, packet count, and peak concurrent sources (`max(UNIQUE_SRCADDR)`), accompanied by time-series graphs of inbound packets per second (PPS) and bits per second (BPS) calculated over flow duration (Figures 3–5).



Figure 3: Inbound Traffic — Top Target Hosts.

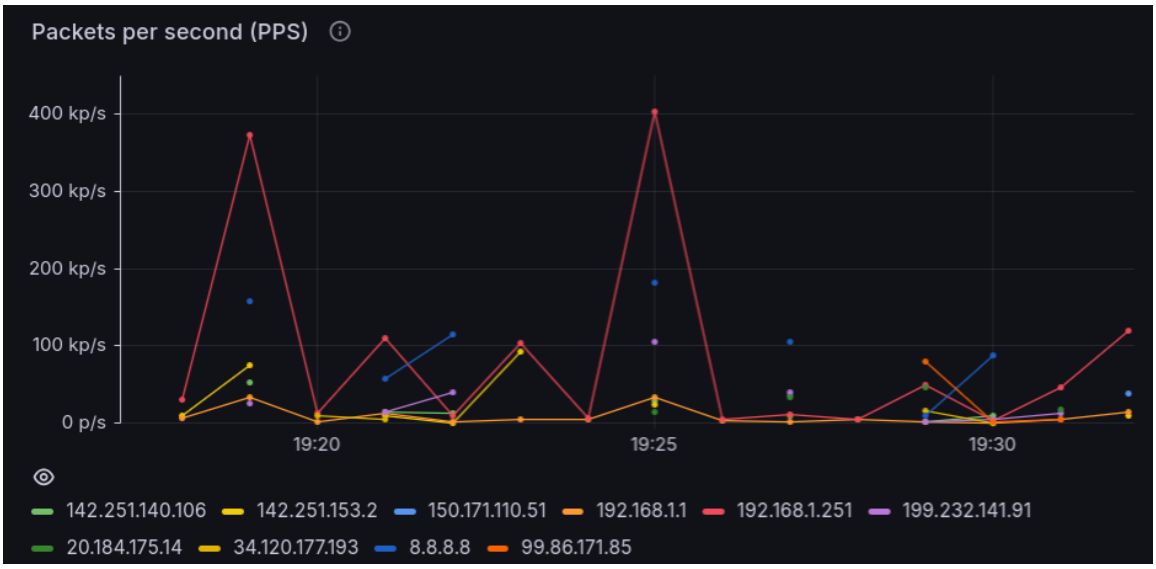


Figure 4: Inbound Traffic — Inbound Packets per Second (PPS).

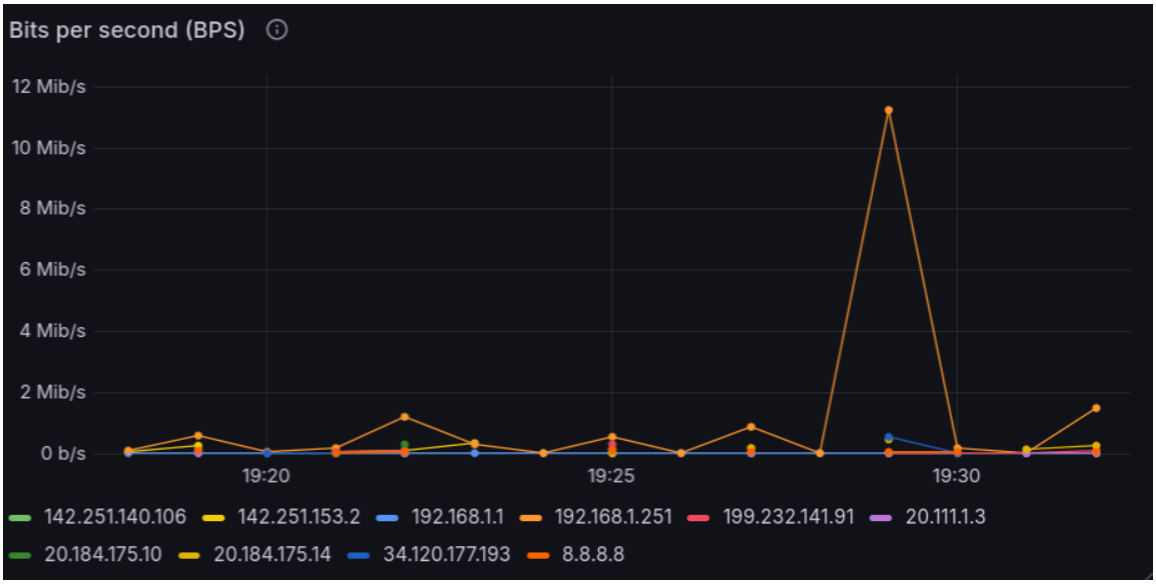


Figure 5: Inbound Traffic — Inbound Bits per Second (BPS).

2. Outbound Traffic Queries `chiba.host_src_view` to identify local top talkers. It correlates emitted traffic volumes with destination IP cardinality (`max(UNIQUE_DSTADDR)`) and contacted destination port cardinality (`max(UNIQUE_DSTPORT)`), tracking outbound bandwidth and packet rates over time (Figures 6–8).

Top Source Hosts

Source Host	Total Volume	Total Packets	Unique Destinations	Targeted Ports
34.120.177.193	1.32 MiB	283	1	1
192.168.1.251	762 KiB	2.84 K	11	3
150.171.110.51	157 KiB	101	1	2
142.251.153.2	104 KiB	249	1	2
150.171.110.56	87.6 KiB	54	1	1
8.8.8.8	75.1 KiB	916	1	1
99.86.171.85	59.3 KiB	88	1	6
192.168.1.1	45.1 KiB	296	2	49
199.232.141.91	42.0 KiB	298	1	3
142.251.140.106	28.3 KiB	138	1	1

Figure 6: Outbound Traffic — Top Source Hosts.

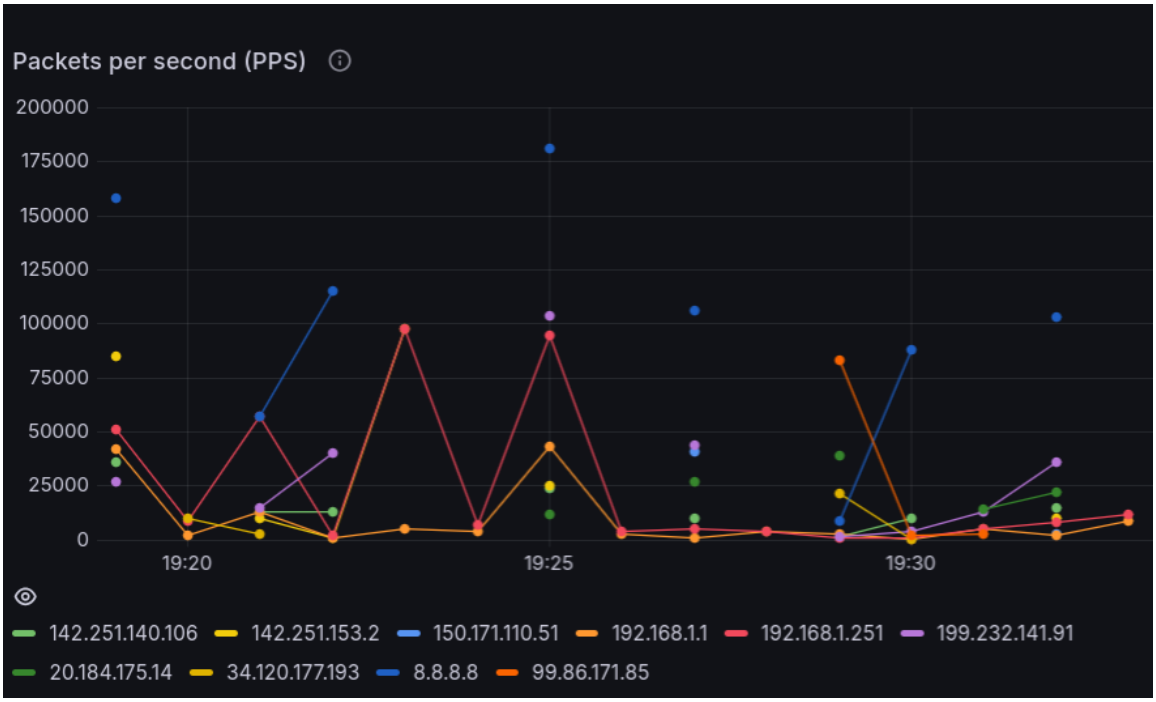


Figure 7: Outbound Traffic — Outbound Packets per Second (PPS).

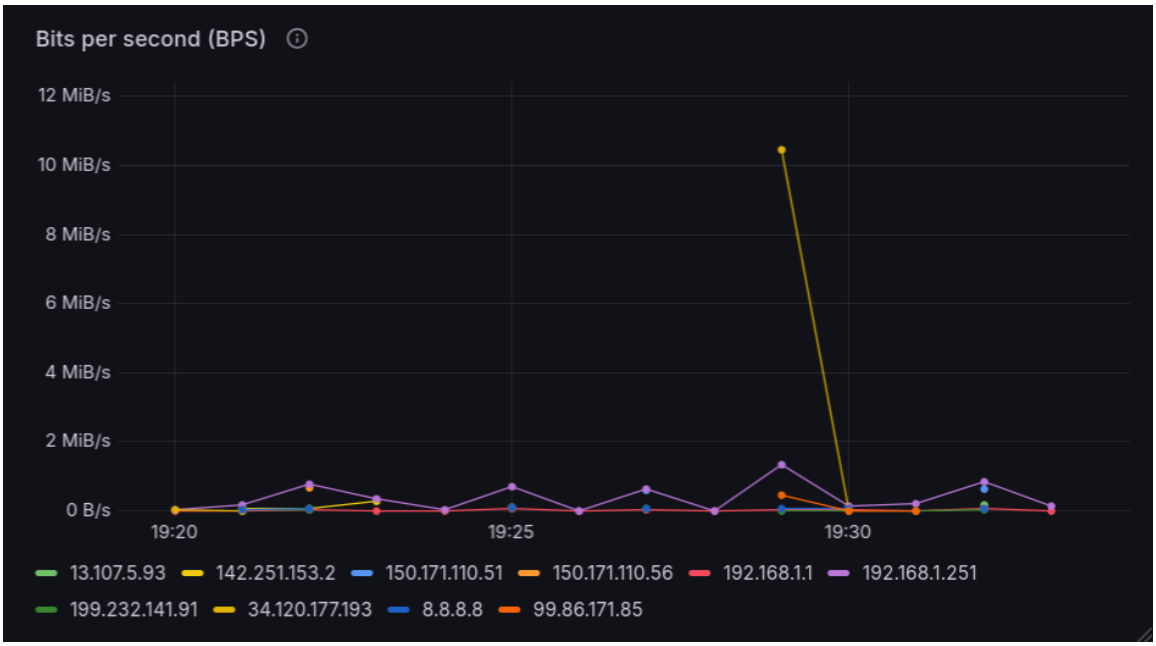


Figure 8: Outbound Traffic — Outbound Bits per Second (BPS).

3. Protocol Distribution Relies on `chiba.service_view` to segment traffic at Layer 4 and the application layer. It displays the percentage share of protocols (TCP, UDP, ICMP/ICMPv6), dominant destination services/ports, and protocol volume persistence over time (Figures 9–11).

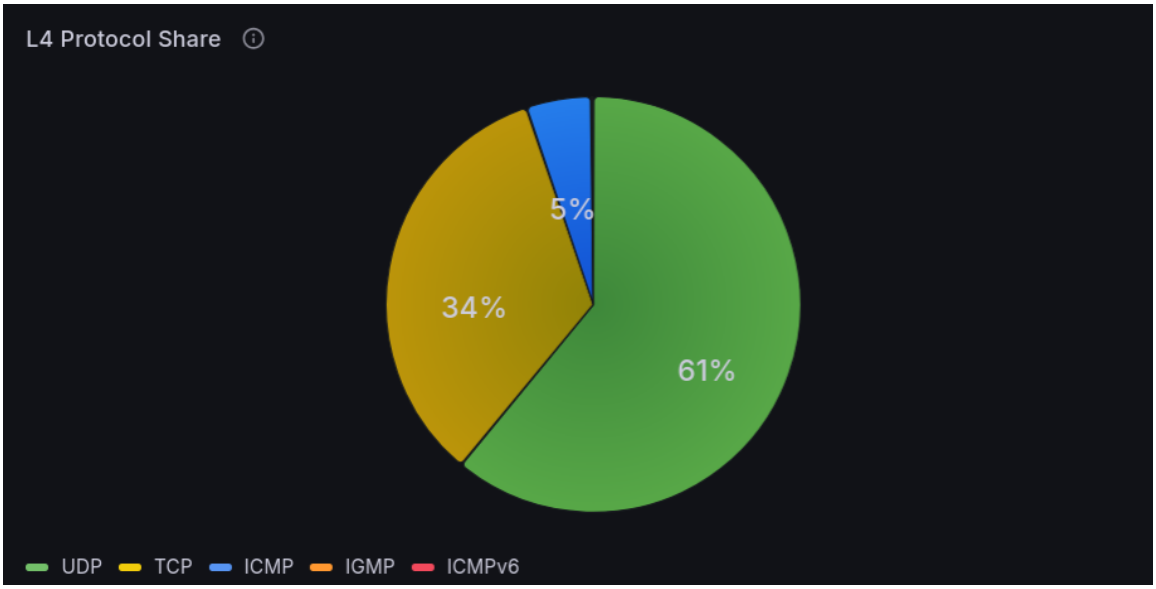


Figure 9: Protocol Distribution — L4 Protocol Share.



Figure 10: Protocol Distribution — Top Active Services / Ports.

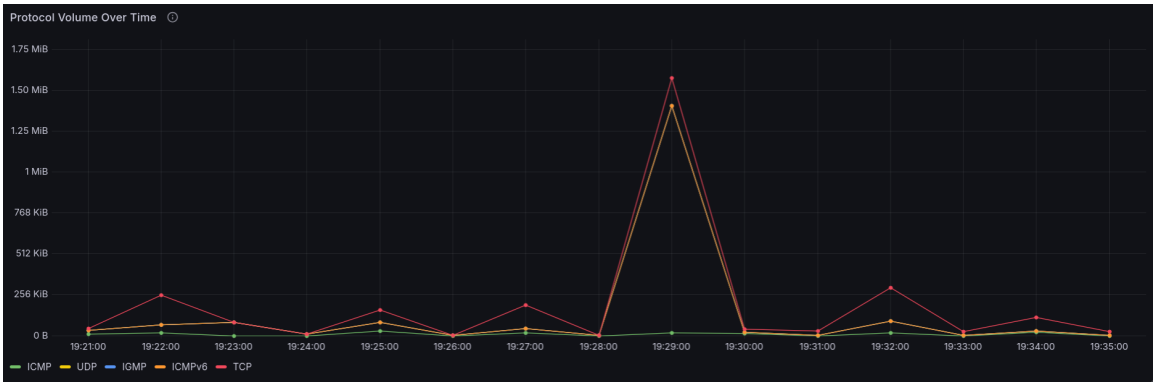


Figure 11: Protocol Distribution — Protocol Volume Over Time.

4. Security Alerts Integrates four real-time status stat panels for immediate counting of hosts involved in malicious activity (DDoS targets, network scanners, port scanners, and high egress anomalies) and two detailed inspection tables for short-term (1-minute) and long-term (24-hour) security incidents (Figures 12–14).

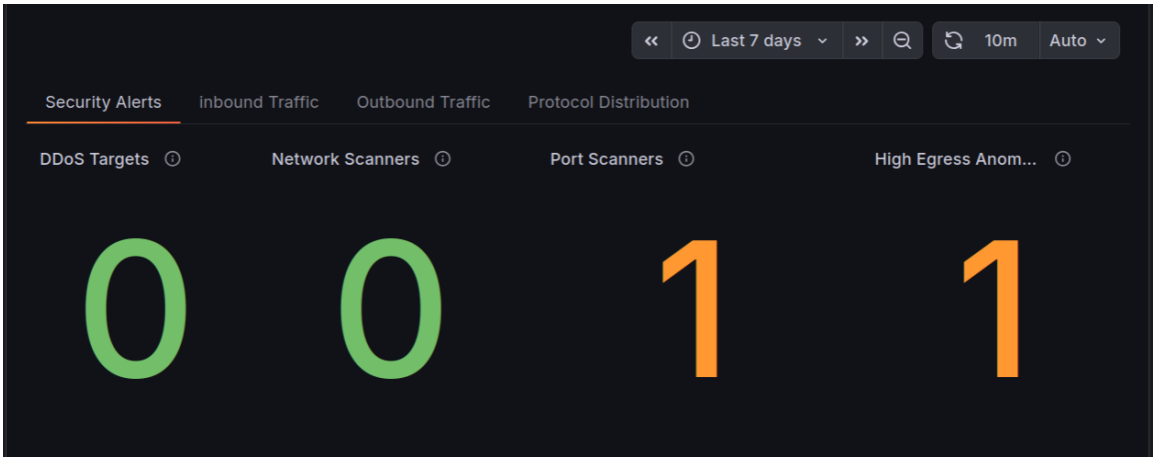


Figure 12: Security Alerts — Real-Time Threat Counters.

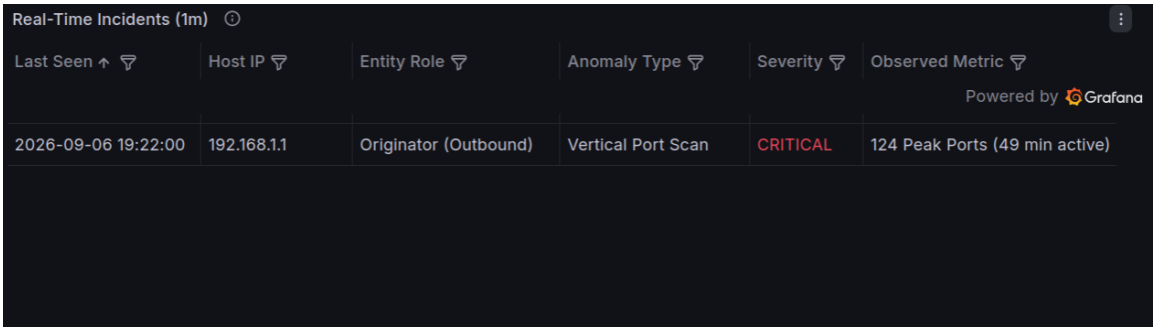


Figure 13: Security Alerts — Real-Time Incidents Table (1-Minute Window).

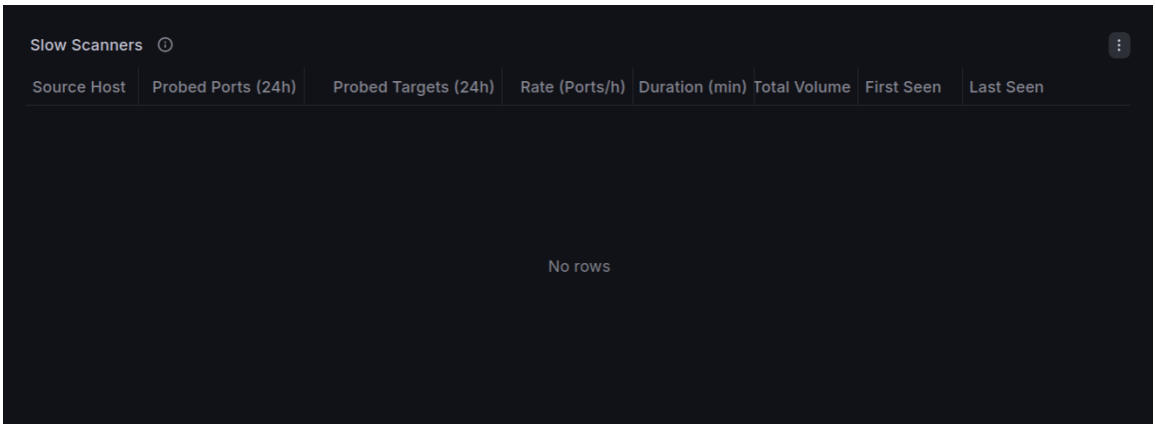


Figure 14: Security Alerts — Slow Scanners Table (24-Hour Sliding Window).

1.4.2 Detection Rules, Thresholds, and Severity

The alerting engine separates the identification of rapid, volumetric anomalies over a 1-minute window from the analysis of slow, distributed reconnaissance anomalies over a 24-hour window.

The numerical thresholds adopted represent a heuristic baseline designed to enable reproducible laboratory testing, without claiming to mirror the exact traffic volumes of a large production network. Thanks to the strict architectural decoupling between the C capture daemon and the analytical layer, all detection logic resides exclusively inside Grafana queries and ClickHouse logical views: any threshold can therefore be recalibrated at runtime without altering the database schema or recompiling the C daemon.

1. Real-Time Detection (Window: 1 Minute) High-frequency anomalies are detected by aggregating `host_dst_view` and `host_src_view` per 1-minute `TIME_BUCKET`. Results are merged into a unified incident table via `UNION ALL`, assigning each event a host role, threat category, and dynamic severity level evaluated through ClickHouse's `multiIf()` function:

- **Distributed Flood / DDoS (Inbound):** Triggered when `UNIQUE_SRCADDR >= 50` toward a single `DSTADDR`. Severity is graded via `multiIf(max(UNIQUE_SRCADDR) >= 200, 'CRITICAL', 'WARNING')`. The convergence of flows from dozens or hundreds of distinct sources within the same minute onto a single target node highlights a distributed volumetric attack (such as a SYN/UDP flood or botnet).
- **Horizontal Network Sweep (Outbound):** Detects local hosts with `UNIQUE_DSTADDR >= 50`, escalating to `CRITICAL` when `max(UNIQUE_DSTADDR) >= 200`. Contacting an anomalous number of distinct destination hosts within 60 seconds identifies horizontal subnet reconnaissance.
- **Vertical Port Scan (Outbound):** Detects source hosts with `UNIQUE_DSTPORT >= 20`, escalating to `CRITICAL` when `max(UNIQUE_DSTPORT) >= 100`. Dispersion across dozens or hundreds of target ports within a single minute isolates vertical port scanning aimed at mapping active services.
- **Excessive Outbound Volume (Outbound):** Monitors flows with `TOTAL_DOCTETS >= 20971520` (20 MiB/min), assigning `CRITICAL` severity starting at `52428800` (50 MiB/min). This rule detects anomalous outbound traffic spikes caused by unauthorized bulk data exfiltration or compromised nodes participating in outbound attacks.

In all cases, the query quantifies incident persistence via `count()`, returning the number of 1-minute buckets in which the anomaly remained active during the selected time window.

2. Extended-Horizon Detection (Window: 24 Hours) Advanced port scanners and stealth reconnaissance tools (such as `nmap` using slow timing templates or the `--scan-delay` flag) deliberately space out probes over time to stay below per-minute thresholds (`UNIQUE_DSTPORT < 20`) and evade real-time detection.

To catch these patterns, the *Slow Scanners* table queries `slow_scanners_view`, which aggregates 1-minute buckets over a rolling 24-hour window (`WHERE TIME_BUCKET >= now() - INTERVAL 24 HOUR`) and applies a joint volume-and-persistence filter:

24-Hour Stealth Scanner Filter (`sql/05_views.sql`)

```
WHERE TIME_BUCKET >= now() - INTERVAL 24 HOUR
GROUP BY SRCADDR
HAVING (UNIQUE_DSTPORT >= 50 OR UNIQUE_DSTADDR >= 100)
   AND ACTIVE_SPAN_MINUTES >= 30
```

- **24-Hour Sliding Window:** The clause `WHERE TIME_BUCKET >= now() - INTERVAL 24 HOUR` merges partial HyperLogLog and min/max states across the entire day, uncovering probes spaced several minutes or hours apart.
- **Temporal Persistence (`ACTIVE_SPAN_MINUTES >= 30`):** Computed via `dateDiff('minute', toDateTime(EXACT_START), toDateTime(EXACT_END))`, isolating sustained reconnaissance while filtering out isolated short bursts or transient false positives.
- **Normalized Scan Rate (`PORTS_PER_HOUR`):** Computed as `round(UNIQUE_DSTPORT / greatest(1, (EXACT_END - EXACT_START) / 3600), 2)`. Using `greatest(1, ...)` prevents division by zero for activities lasting less than an hour, normalizing probe frequency on an hourly basis to estimate the probing rate of the offending host.

2 Prerequisites and Execution Instructions

2.1 System Requirements and Dependency Installation

The software architecture relies on four core components: the C daemon (`chiba`), `softflowd`, ClickHouse, and Grafana. The following installation instructions refer to Debian/Ubuntu-based distributions.

2.1.1 Development Toolchain, Libraries, and Capture Probe

To compile the daemon and enable NetFlow capture and decoding, install the C build toolchain, `libcurl` development headers, and `softflowd`:

Toolchain and Probe Installation

```
sudo apt update
sudo apt install build-essential gcc make libcurl4-openssl-dev softflowd
```

2.1.2 ClickHouse Installation (Server and Client)

ClickHouse can be installed using the official installation script, which sets up the binaries and registers the `systemd` service:

ClickHouse Server & Client Installation

```
# Download and run the official ClickHouse installer
curl https://clickhouse.com/ | sh
sudo ./clickhouse install

# Start the service for the current session
sudo systemctl start clickhouse-server

# (Optional) Enable automatic startup at boot:
# sudo systemctl enable clickhouse-server

# Verify service status and installed version
clickhouse-client --query "SELECT version();"
```

2.1.3 Grafana and ClickHouse Plugin Installation

Grafana OSS and ClickHouse Datasource Plugin Installation

```
# Configure official APT repository and install Grafana OSS
sudo apt install apt-transport-https software-properties-common wget
sudo mkdir -p /etc/apt/keyrings/
wget -q -O - https://apt.grafana.com/gpg.key | gpg --dearmor \
  | sudo tee /etc/apt/keyrings/grafana.gpg > /dev/null
echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg] https://apt.grafana.com stable main" \
  | sudo tee /etc/apt/sources.list.d/grafana.list
sudo apt update && sudo apt install -y grafana

# Install the official ClickHouse datasource plugin for Grafana
sudo grafana-cli plugins install grafana-clickhouse-datasource

# Start the Grafana server daemon
sudo systemctl start grafana-server

# (Optional) Enable at boot:
# sudo systemctl enable grafana-server
```

2.1.4 Test Traffic Generation Tools

To run the security validation tests and traffic simulations described in Section 3:

Traffic Simulation Utilities

```
sudo apt install nmap hping3 netcat-openbsd tcpreplay
```

2.2 Compiling the CHIBA Daemon

Building the executable is automated by the `Makefile` in the project root. Running `make` compiles all source modules in `src/`, creates the intermediate object directory `build/`, and produces the compiled binary at `bin/chiba`:

Compiling and Cleaning

```
# Compile the daemon
make

# Remove object files and compiled binaries
make clean
```

2.3 Database Initialization and Management

The `Makefile` includes dedicated targets to sequentially execute all SQL schema scripts located in `sql/` (`00_init.sql` → `05_views.sql`), provisioning the `chiba` database, base table, intermediate state tables, materialized views, and logical views:

Database Schema Management

```
# Initialize the full database schema:
make db-init

# Full database reset (Drop and Re-init):
# Useful during testing to clear all table states.
# Prompts for explicit confirmation ('yes') to prevent accidental data loss:
make db-reset
```

2.4 Daemon Configuration and Execution

HTTP connection parameters for ClickHouse are defined in `chiba.conf` in the project root:

`chiba.conf`

```
CH_HOST=http://127.0.0.1:8123
CH_DB=chiba
CH_TABLE=ingest_flows
```

These parameters can be overridden at runtime using environment variables of the same name (`CH_HOST`, `CH_DB`, `CH_TABLE`).

As described in Section 1.2, the daemon supports two mutually exclusive execution modes:

Running CHIBA

```
# Live capture on a network interface
# (requires root privileges for raw sockets in softflowd)
sudo ./bin/chiba -i <network interface name>

# Offline analysis of a static PCAP file
./bin/chiba -r ./pcap/DHCPv6.pcap
```

2.5 Grafana Dashboard Configuration

1. Open the Grafana web interface at `http://localhost:3000` (default credentials: username `admin`, password `admin`).
2. From the side menu, navigate to **Connections** → **Data sources** → **Add data source** and select **ClickHouse**:

- **Server address:** `127.0.0.1`
- **Server port:** `8123`
- **Default database:** `chiba`
- Click **Save & test** to verify connectivity with the DBMS.

3. Navigate to **Dashboards** → **New** → **Import**, click **Upload dashboard JSON file**, and select `grafana/chiba_dashboard.json`.

3 Testing

3.1 Fast Port Scan

The objective of this test is to verify the detection of an aggressive vertical port scan against a single target host, exceeding the alarm threshold of 20 distinct destination ports within a 1-minute bucket.

The command used to generate the anomaly on the loopback interface is:

Fast Vertical Port Scan Simulation

```
sudo nmap -sS -T4 -p 1-150 127.0.0.1
```

After the flow flush interval (60 seconds), querying `chiba.host_src_view` directly confirms proper flow aggregation:

ClickHouse Verification Query

```
clickhouse-client --query "
SELECT
    SRCADDR,
    UNIQUE_DSTPORT,
    PROT
FROM chiba.host_src_view
WHERE UNIQUE_DSTPORT >= 20;
"
```

Output:

```
::ffff:127.0.0.1    150    6
```

In the Grafana dashboard, the incident is immediately reflected in the alerting panels:

- The **Port Scanners** counter increments to `1`.
- **Real-Time Incidents (1m) Table:** Host `127.0.0.1` is logged with anomaly type `Vertical Port Scan` and severity `CRITICAL` (having exceeded 100 ports).

3.2 Network Sweep (Horizontal Scan)

The objective of this test is to verify the detection of horizontal subnet reconnaissance aimed at mapping active hosts, exceeding the alarm threshold of 50 distinct destination addresses within 1 minute.

On Linux, the entire `127.0.0.0/8` subnet responds on the loopback interface. The command used to generate TCP probes toward 65 distinct IP addresses is:

Horizontal Network Sweep Simulation

```
sudo nmap -sS -T4 -p 80 127.0.0.1-65
```

At the end of the aggregation interval, querying `chiba.host_src_view` confirms that the threshold has been exceeded:

ClickHouse Verification Query

```
clickhouse-client --query "
SELECT
    SRCADDR,
    UNIQUE_DSTADDR,
    PROT
FROM chiba.host_src_view
WHERE UNIQUE_DSTADDR >= 50;
"
```

Output:

```
::ffff:127.0.0.1    65    6
```

In the dashboard, the event is promptly displayed in the Security Alerts row:

- The **Network Scanners** counter increments to `1`.
- **Real-Time Incidents (1m) Table:** Host `127.0.0.1` is reported with role `Originator (Outbound)`, anomaly `Horizontal Network Sweep`, and severity `WARNING` (severity escalates to `CRITICAL` above 200 destinations).

3.3 Distributed Flood / DDoS

The objective of this test is to verify the system's ability to intercept converging volumetric attacks, detecting when a single target host receives traffic from at least 50 distinct source IPs within 60 seconds.

The command uses `hping3` to transmit 80 TCP SYN packets toward the loopback address using spoofed random source IP addresses via the `--rand-source` flag:

Distributed SYN Flood Simulation

```
sudo hping3 -c 80 -d 10 -S -p 80 --rand-source 127.0.0.1
```

Querying `chiba.host_dst_view` highlights the concentration of concurrent sources on the target:

ClickHouse Verification Query

```
clickhouse-client --query "
SELECT
    DSTADDR,
    UNIQUE_SRCADDR
FROM chiba.host_dst_view
WHERE UNIQUE_SRCADDR >= 50;
"
```

Output:

```
::ffff:127.0.0.1    80
```

The incident is immediately reflected in the monitoring panels:

- The **DDoS Targets** counter increments to `1`.
- **Real-Time Incidents (1m) Table:** Target `127.0.0.1` is recorded with role `Target (Inbound)`, anomaly `Distributed Flood`, and severity `WARNING`.

3.4 Anomalous Egress Volume (Data Exfiltration)

The objective of this test is to verify alerting on anomalous outbound traffic flows exceeding the volumetric threshold of 20 MiB (20,971,520 bytes) within 1 minute.

The test spawns a local listener on port 9999 and transfers 30 MiB of random data over loopback using `dd` and `nc`:

Bulk Egress Transfer Simulation

```
# Start local listener in background
nc -l -p 9999 > /dev/null &

# Bulk transfer of 30 MiB
dd if=/dev/urandom bs=1M count=30 | nc -w 1 127.0.0.1 9999
```

Querying `chiba.host_src_view` confirms accurate quantification of the emitted byte volume:

ClickHouse Verification Query

```
clickhouse-client --query "
SELECT
    SRCADDR,
    formatReadableSize(TOTAL_DOCTETS) AS volume,
    TOTAL_DOCTETS
FROM chiba.host_src_view
WHERE TOTAL_DOCTETS >= 20971520;
"
```

Output:

```
::ffff:127.0.0.1    30.00 MiB    31457280
```

In the dashboard, the volumetric anomaly is displayed with automatic unit formatting:

- The **High Egress Anomalies** counter increments to `1`.

- **Real-Time Incidents (1m) Table:** A record is generated for host `127.0.0.1` with type `Excessive Outbound Volume` and severity `WARNING` (escalating to `CRITICAL` above 50 MiB).

3.5 Dual-Stack IPv6 Support from PCAP Trace

The objective of this test is to validate offline processing of static PCAP traces and demonstrate that the NetFlow v9 parser accurately handles native 128-bit IPv6 addresses without memory corruption or endianness misalignment.

The command launches the daemon in offline mode on the IPv6 test capture:

Offline IPv6 PCAP Ingestion

```
./bin/chiba -r ./pcap/DHCPv6.pcap
```

In PCAP mode, flows are recorded with the current system timestamp (`time(NULL)`), making them immediately visible in the Grafana dashboard without having to adjust the time picker to historical dates.

Querying `chiba.ingest_flows` verifies the presence of native IPv6 addresses lacking the IPv4-mapped prefix (`::ffff:`):

ClickHouse Native IPv6 Verification Query

```
clickhouse-client --query "
SELECT
    toDateTime(START_TIME) AS time,
    IPv6NumToString(SRCADDR) AS src,
    IPv6NumToString(DSTADDR) AS dst,
    SRCPORT,
    DSTPORT,
    PROT
FROM chiba.ingest_flows
WHERE notLike(IPv6NumToString(SRCADDR), '::ffff:%')
LIMIT 5;
"
```

Output:

```
2026-09-20 16:50:00  fe80::a00:27ff:fed4:10bb  fe80::a00:27ff:fefe:8f95  547  546  17
2026-09-20 16:50:00  fe80::a00:27ff:fed4:10bb  ff02::16                  0    0    255
2026-09-20 16:50:00  fe80::a00:27ff:fed4:10bb  ff02::1:fffe:8f95         0    0    58
2026-09-20 16:50:00  fe80::a00:27ff:fefe:8f95  fe80::a00:27ff:fed4:10bb  0    0    58
2026-09-20 16:50:00  fe80::a00:27ff:fefe:8f95  ff02::1:2                 546  547  17
```

In the *Protocol Distribution* row, IPv6 flows are aggregated accurately:

- The **L4 Protocol Share** pie chart properly separates UDP and ICMPv6 (protocol 58) traffic.
- The service volume table records DHCPv6 ports (`546` and `547`) and Neighbor Discovery messages.

3.6 Slow Port Scan (24-Hour Window)

The objective of this test is to demonstrate the effectiveness of the extended-horizon detection engine: a low-frequency stealth reconnaissance diluted over time evades per-minute checks but is caught by the 24-hour sliding window view.

A diluted scan (for example, executed via `nmap -T1` or by setting an explicit probe delay with `--scan-delay 15s`) keeps the number of probes per individual minute consistently below the fast threshold (`UNIQUE_DSTPORT < 20`).

While `host_src_view` raises no alerts for any single 1-minute bucket, querying the 24-hour view `slow_scanners_view` correlates the cumulative probes once persistence and cardinality criteria (`ACTIVE_SPAN_MINUTES >= 30` and `UNIQUE_DSTPORT >= 50`) are met:

ClickHouse 24-Hour Slow Scanner Verification Query

```
clickhouse-client --query "
SELECT
    SRCADDR,
    UNIQUE_DSTPORT,
    ACTIVE_SPAN_MINUTES,
    PORTS_PER_HOUR
FROM chiba.slow_scanners_view;
"
```

Output:

```
::ffff:127.0.0.1    60    35    102.86
```

This dual-horizon behavior is clearly reflected in the dashboard:

- **Real-Time Incidents (1m) Table:** Zero alerts generated for individual 60-second buckets (confirming successful evasion of real-time per-minute thresholds).
- **Slow Scanners (24h Window) Table:** The offending host is surfaced along with its normalized `Rate (Ports/h)` metric, exposing the stealth reconnaissance activity.

4 Conclusions and Future Work

We can now draw several conclusions regarding the project.

The architecture is built around a core principle: a network monitoring system must operate autonomously, without requiring manual intervention to aggregate data or correlate anomalies that can be identified algorithmically in software. For this reason, metric computation and anomalous host identification occur entirely automatically inside ClickHouse.

At the same time, to prevent alert fatigue from causing operators to overlook real threats, the system clearly separates statistical anomaly warnings (`WARNING`) from severe, high-confidence incidents (`CRITICAL`), and combines a short-term 1-minute detection window with a 24-hour sliding window—filtering out normal network fluctuations while surfacing slow, persistent scans. Finally, to ensure critical alerts never go unnoticed, security indicators are placed at the top visual hierarchy of the dashboard to immediately capture operator attention.

Below are several directions for future development:

- **Docker Compose Orchestration:** Coordinate the deployment, networking, and provisioning of ClickHouse, Grafana, `softflowd`, and CHIBA through a single `docker-compose.yml` specification.
- **Structured Logging System in C:** Replace formatted terminal print statements with a level-based structured logging subsystem (`DEBUG`, `INFO`, `ERROR`), which will be especially valuable as the codebase grows.
- **External Alert Notifications:** Dispatch automated notifications via email, Telegram, Discord, or webhooks whenever `CRITICAL` incidents are triggered.
- **Dynamic Baseline Thresholds:** Complement static heuristic thresholds with statistical baselines computed over historical network traffic, continuously adapting anomaly boundaries to the monitored environment.

5 References and Bibliography

- Brian "Beej Jorgensen" Hall, *Beej's Guide to Network Programming — Using Internet Sockets*: https://beej.us/guide/bgnet/html/
- Michael Kerrisk, *The Linux Programming Interface*: No Starch Press
- Luca Deri, *Network Monitoring in Practice*: https://luca.ntop.org/gr2026/tm2026.pdf
- Cisco Systems, *RFC 3954 — Cisco Systems NetFlow Services Export Version 9*: https://datatracker.ietf.org/doc/html/rfc3954
- IANA (Internet Assigned Numbers Authority), *Protocol Numbers*: https://www.iana.org/assignments/protocol-numbers/protocol-numbers.xhtml
- ClickHouse Official Documentation: https://clickhouse.com/docs
- Grafana Official Documentation: https://grafana.com/docs